

CmpE 260 – Principles of Programming Languages

Project 1 Design Report: Mini Programming Language Interpreter

Emirhan Yakup Altuntaş, 2023400084
Efe Dikilitaş, 2023400045

Spring 2026

1 Language Overview

This project implements an interpreter for a small expression-oriented programming language. The language includes integer and boolean values, arithmetic and logical operators, conditional expressions, functions, closures, assignment, printing, and block evaluation. The implementation also includes all four bonus features: strings, while loops, lists, and selectable static/dynamic scoping.

The interpreter is organized as a conventional pipeline. Source code is first converted into a token list by the lexer. The parser then converts the token stream into an abstract syntax tree. Finally, the evaluator executes the AST using an environment-based runtime model. This separation keeps lexical, syntactic, and runtime concerns in different parts of the implementation, which makes the interpreter easier to debug and extend.

2 Scoping Decision

The default scoping mode is **static scoping**, also called **lexical scoping**. In static scoping, a free variable inside a function is resolved using the environment that was active when the function was defined. Therefore, the meaning of a variable can be determined by reading the source code structure rather than by inspecting the runtime call stack.

Static scoping was selected as the default because it matches the behavior of many modern programming languages and works naturally with closures. A closure stores both the function code and the definition-time environment. When the closure is called later, free variables are found by following the parent chain from that captured environment.

Dynamic scoping is also implemented as bonus feature B4. In dynamic scoping, a free variable is resolved from the caller's environment instead of the definition environment. This is useful for demonstrating the difference between the two scoping rules, but it makes program behavior harder to predict because the same function may produce different results depending on where it is called.

```
let x = 1;
let f = fun () -> x end;
let test = fun () ->
  let x = 99;
```

```
f()
end;
print(test());
```

In static mode, this program prints 1, because `f` captures the environment where `x` was 1. In dynamic mode, it prints 99, because the lookup for `x` follows the call-site environment inside `test`. The command-line interface supports both modes:

```
python interpreter.py program.txt
python interpreter.py --scope static program.txt
python interpreter.py --scope dynamic program.txt
```

3 Evaluation Strategy

The interpreter uses **call-by-value** evaluation. Before a function body starts executing, every argument expression is evaluated to a value. Those values are then bound to the corresponding parameter names in the function call frame.

This strategy is simple and predictable: each argument is evaluated exactly once before the function is entered. It also matches the strict evaluation model used by languages such as Python, Java, and C. The tradeoff is that unused arguments are still evaluated. Under call-by-name, an unused argument could be skipped, but that would require representing unevaluated expressions and their environments as thunks.

```
let ignore = fun (x) -> 42 end;
let side_effect = fun () -> print(999); 0 end;
print(ignore(side_effect()));
```

With call-by-value, `side_effect()` is evaluated before `ignore` is called. Therefore, the program prints 999 first and then 42. With call-by-name, the argument would not be evaluated because `x` is never used in the body of `ignore`.

4 Abstract Syntax Tree Design

The parser converts the token stream into an abstract syntax tree. Each AST node represents one syntactic form in the language. Literal nodes represent values, operator nodes represent computations, statement nodes represent side effects, and compound nodes represent functions, calls, blocks, loops, and lists.

Node Type	Fields	Purpose
NumberLit	value	Integer literal.
BoolLit	value	Boolean literal.
StringLit	value	B1 string literal with decoded escapes.
ListLit	elements	B3 list literal such as [] or [1, 2, 3].
Identifier	name	Variable reference.
BinOp	op, left, right	Arithmetic, comparison, boolean, and equality operations.

UnaryOp	op, operand	Unary minus and logical not.
IfExpr	cond, then_branch, else_branch	Expression-level conditional.
FunExpr	params, body	Anonymous function definition.
CallExpr	func, args	Function calls and built-in calls.
IndexExpr	collection, index	B3 list indexing expression such as <code>xs[i]</code> .
LetStmt	name, value	New binding in the current environment frame.
AssignStmt	name, value	Mutation of an existing binding in the environment chain.
PrintStmt	expr	Output statement.
WhileStmt	cond, body	B2 while loop statement.
Block	statements, final_expr	Sequence of statements with an optional final expression.

5 Closure Representation

A function value is represented as a `Closure` object. A closure contains the parameter list, the function body AST, and the environment that was active when the `fun` expression was evaluated. This environment reference is the key mechanism that implements static scoping.

```
class Closure:
    def __init__(self, params, body, env):
        self.params = params
        self.body = body
        self.env = env
```

When a function is called in static mode, the interpreter creates a fresh call frame whose parent is `closure.env`. This means the function body can access parameters in the new frame and free variables in the captured definition environment. In dynamic mode, the fresh call frame instead uses the current call-site environment as its parent. This single parent-selection point is what separates the two scoping modes.

6 Environment Management

The runtime environment is implemented as a linked chain of frames. Each `Environment` object stores a dictionary of local bindings and a parent pointer. Lookup searches the current frame first and then walks upward through the parent chain until it finds the requested name.

The distinction between `let` and assignment is important. A `let` statement creates or replaces a binding in the current frame. An assignment searches upward and updates the nearest existing binding. This is necessary for mutable closures, such as counters, because the inner function must update the captured `count` variable rather than create a new local one.

```
let make_counter = fun (start) ->
    let count = start;
    fun () ->
        count = count + 1;
        count
    end
```

```
end;
let c = make_counter(0);
print(c());
print(c());
```

The two calls print 1 and 2. The variable `count` lives in the environment frame created by `make_counter`. The returned inner function keeps that frame alive through its closure, so every later call updates the same captured binding.

7 Required Static-Scoping Trace

The following program demonstrates how closure environments and assignment interact under static scoping.

```
let x = 1;
let f = fun (a) ->
  let y = a + x;
  fun () ->
    x = x + 1;
    y + x
  end
end;
let g = f(10);
print(g());
print(g());
```

Step 1. The global frame `G` is created. After the first declaration, `G` contains `x -> 1`. After defining `f`, `G` also contains `f -> Closure(params=[a], body=..., env=G)`. The closure for `f` captures `G` because `f` is defined at the top level.

Step 2. Calling `f(10)` creates frame `F1` with parent `G` and binding `a -> 10`. Inside `f`, `y` is evaluated as `a + x`. The name `a` is found in `F1` and `x` is found in `G`, so `y` becomes 11. The inner `fun` expression creates closure `C` with `env=F1`. After `f` returns, `g` is bound to `C` in `G`.

Step 3. The first call `g()` creates frame `F2` whose parent is `F1`. The assignment `x = x + 1` searches `F2`, then `F1`, then `G`, where `x` is found. Therefore `x` changes from 1 to 2 in `G`. The expression `y + x` finds `y=11` in `F1` and `x=2` in `G`, so the result is 13.

Step 4. The second call `g()` creates another fresh frame `F3` whose parent is still `F1`. The assignment again updates `x` in `G`, changing it from 2 to 3. The value `y` remains 11 in `F1`. Therefore `y + x` becomes `11 + 3 = 14`. The complete output is 13 followed by 14.

8 Bonus Features

8.1 B1 – Strings

String literals are written in double quotes. The lexer recognizes escape sequences such as newline, tab, backslash, and escaped quote. The `+` operator performs integer addition when both operands are integers and string concatenation when both operands are strings. The built-in `length(s)` returns the length of a string.

8.2 B2 – While Loops

While loops use the syntax `while condition do block end`. They are statements rather than expressions. The condition must evaluate to a boolean; otherwise the evaluator raises a runtime type error.

8.3 B3 – Lists

Lists are supported through `ListLit` and `IndexExpr` AST nodes. The parser accepts empty lists, non-empty lists, nested lists, and postfix indexing. The evaluator checks that the indexed value is a list, the index is an integer, and the index is within bounds. The `length` built-in works on both strings and lists, and `append(list, value)` mutates the given list and returns the same list.

8.4 B4 – Static/Dynamic Scoping

The interpreter supports an optional `-scope` flag. Static is the default and uses the closure definition environment. Dynamic uses the call-site environment. The parser and AST are identical in both modes; only the parent environment chosen during function calls changes.

9 Error Handling and Type Safety

The interpreter reports syntax and runtime errors through `RuntimeError` messages printed to `stderr` and exits with code 1. Important runtime checks include division by zero, undefined variables, assignment to undefined variables, arity mismatches, calling non-functions, invalid while/if conditions, invalid list indexing, and list index bounds errors.

Python treats `bool` as a subclass of `int`, so the evaluator uses exact type checks instead of `isinstance` for integer and boolean operations. This prevents invalid programs such as `true + 1`, `true < 3`, or `-true` from being accepted accidentally.

10 Challenges Encountered

The first challenge was recursive `let`-bindings. If the right-hand side of a `let` is evaluated before the name exists in the environment, recursive functions cannot refer to themselves. The solution is to insert the name with a placeholder first, evaluate the function expression, and then update the placeholder with the closure. Because the closure stores the environment by reference, the final binding becomes visible to recursive calls.

The second challenge was distinguishing assignment from expression statements. Both can begin with an identifier. The parser resolves this by parsing an expression first and then converting it to an `AssignStmt` only if the next token is `=` and the parsed expression is a plain `Identifier`. Otherwise, the expression remains a normal expression statement or block final expression.

The third challenge was adding B3 without disrupting function calls. Calls and indexing are both postfix operators, so the parser uses a postfix parsing stage that repeatedly accepts either function-call syntax or list-index syntax. This allows combinations such as `f(x)[0]` or `xs[length(xs)-1]`.

The final challenge was adding B4 cleanly. Instead of duplicating the evaluator, the implementation uses a global scope mode selected by the command line. During a function call, the interpreter chooses either `closure.env` for static scoping or the current environment for dynamic scoping.

11 Testing Summary

The final version was tested with the required example categories and additional tests for each bonus feature. Core examples cover arithmetic, comparisons, variables, conditionals, recursion, closures, static scoping, and higher-order functions. Bonus tests cover string concatenation and length, while-loop iteration, list literals and indexing, append mutation, nested lists, static/dynamic scoping differences, and expected runtime errors.

The final implementation claim is: core project features F1–F10, call-by-value evaluation, static lexical scoping by default, B1 strings, B2 while loops, B3 lists, and B4 selectable static/dynamic scoping.